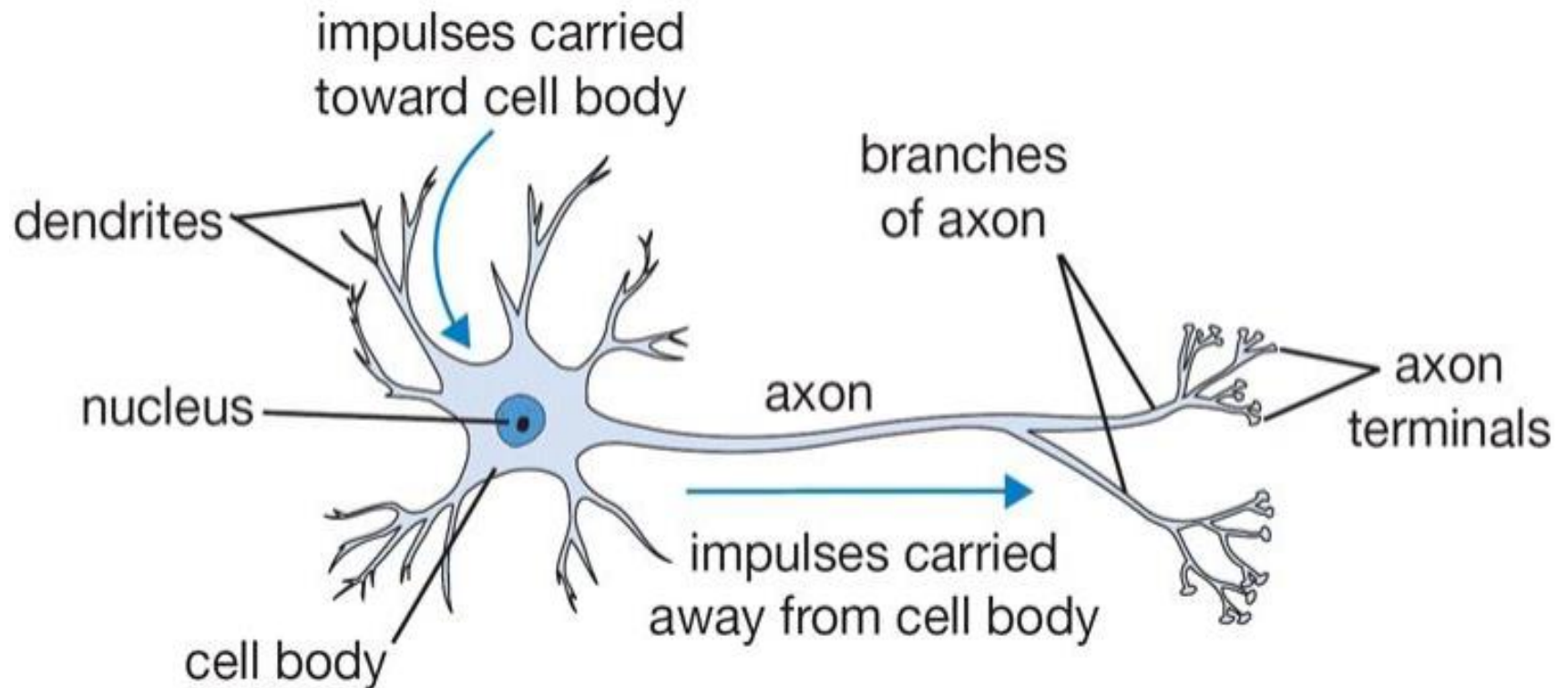


Perceptron Model

Inspiration: Biological Neuron

<http://neuroscience.uth.tmc.edu/s1/chapter01.html>



Perceptron model: regression

Problem

- Assume you got a new job offer. How do you make decision of to take or reject offer?
 - Inputs
 - Salary is good enough(rate it 1 to 5)
 - Work environment(1 to 5 scale)
 - Flexibility(1 o 5 scale)
 - Company rating(1 to 5 scale)
 - The importance of each of the above inputs
 - Give weights for each of the above inputs and also threshold for yes decision

Perceptron: model for regression

Objective function for Learning

- Objective function = loss/error function +
regularizer function
- Squared loss is the most intuitive loss function for regression pattern learning
- Squared loss in formal notation:

Represent $w_0 + w_1x_1$ in vector notation as wx

$$E(w) = \frac{1}{2} \sum_{n=1}^N (y^{(n)} - (wx)^{(n)})^2$$

GD Algorithm with squared loss

- Assume some random weights and random bias for perceptron
- Repeat the following until the error is below threshold or maximum number of epochs reached:
 - a. Pick a training sample $(x^{(n)}, y^{(n)})$ and compute the output of perceptron, $o^{(n)}$
 - b. Update each weight, w_j , of the perceptron as follows:

$$w_j = w_j - \eta \frac{\partial E}{\partial w_j}$$

$$w_j = w_j + \eta \sum_{n=1}^N (y^{(n)} - o^{(n)}) x_j^{(n)}$$

Note: keep $x_j^{(n)} = 1$ for w_o (bias input)

Perceptron model: classification

Problem

- Assume you got a new job offer. How do you make decision of to take or reject offer?
 - Inputs
 - Salary is good enough(rate it 1 to 5)
 - Work environment(1 to 5 scale)
 - Flexibility(1 o 5 scale)
 - Company rating(1 to 5 scale)
 - The importance of each of the above inputs
 - Give weights for each of the above inputs and also threshold for yes decision

Perceptron: model for classification

Objective function for Learning

- Objective function = loss/error function +
regularizer function
- Squared loss is the simplest(but not best) loss function for classification pattern learning
- Squared loss in formal notation:

Represent $w_0 + w_1x_1$ in vector notation as wx

$$E(w) = \frac{1}{2} \sum_{n=1}^N (y^{(n)} - g(wx)^{(n)})^2$$

GD Algorithm with squared loss

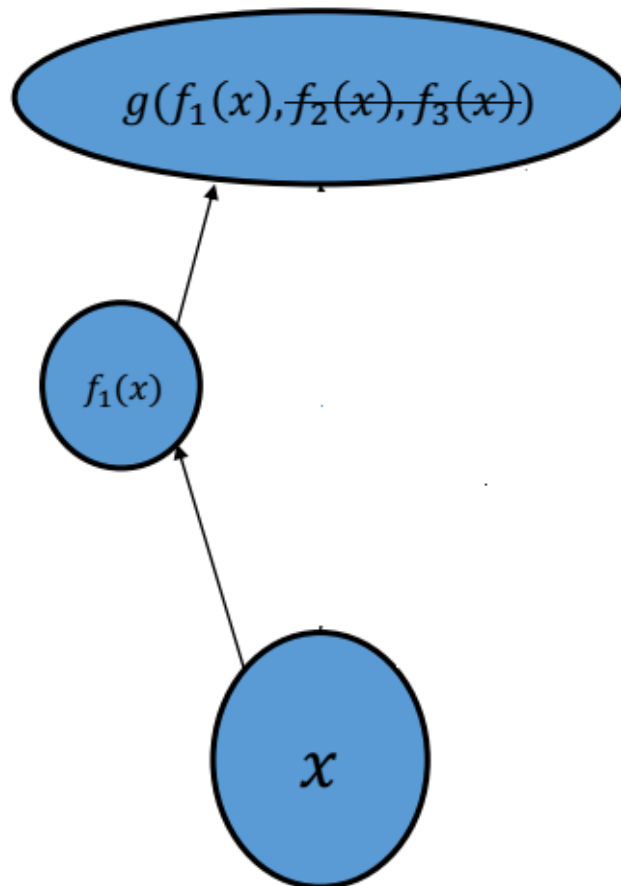
- Assume some random weights and random bias for perceptron
- Repeat the following until the error is below threshold or maximum number of epochs reached:
 - a. Pick a training sample $(x^{(n)}, y^{(n)})$ and compute the output of perceptron, $o^{(n)}$
 - b. Update each weight, w_j , of the perceptron as follows:

$$w_j = w_j - \eta \frac{\partial E}{\partial w_j}$$

$$w_j = w_j + \eta \sum_{n=1}^N (y^{(n)} - o^{(n)}) x_j^{(n)} g'(a)^{(n)}$$

Note: keep $x_j^{(n)} = 1$ for w_o (bias input)

Computing EW: Chain Rule



$$\frac{\partial g}{\partial x} = \frac{\partial g}{\partial f_1} \frac{\partial f_1}{\partial x}$$

Error derivatives of weights

$$\frac{\partial E}{\partial w_j} = \frac{\partial E}{\partial o} \frac{\partial o}{\partial a} \frac{\partial a}{\partial w_j}$$

$$\frac{\partial a}{\partial w_j} = \frac{\partial \sum_k w_k x_k}{\partial w_j} = x_j$$

$$\frac{\partial o}{\partial a} = g'(a)$$

Error derivatives of weights

$$\frac{\partial E}{\partial o} = \frac{1}{2} \frac{\partial}{\partial o^{(n)}} (y - o)^2$$

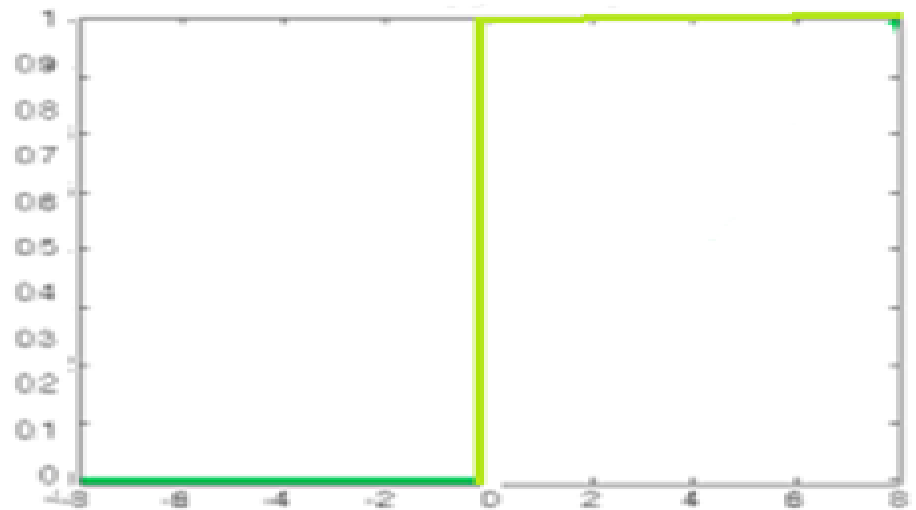
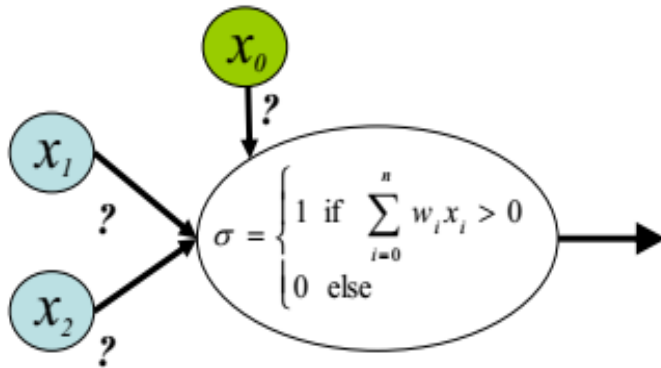
$$\frac{\partial E}{\partial o} = \frac{1}{2} 2 * (y - o) * \frac{\partial}{\partial o^{(n)}} (y - o)$$

$$\frac{\partial E}{\partial o} = (y - o) * -1$$

Issues of perceptron model

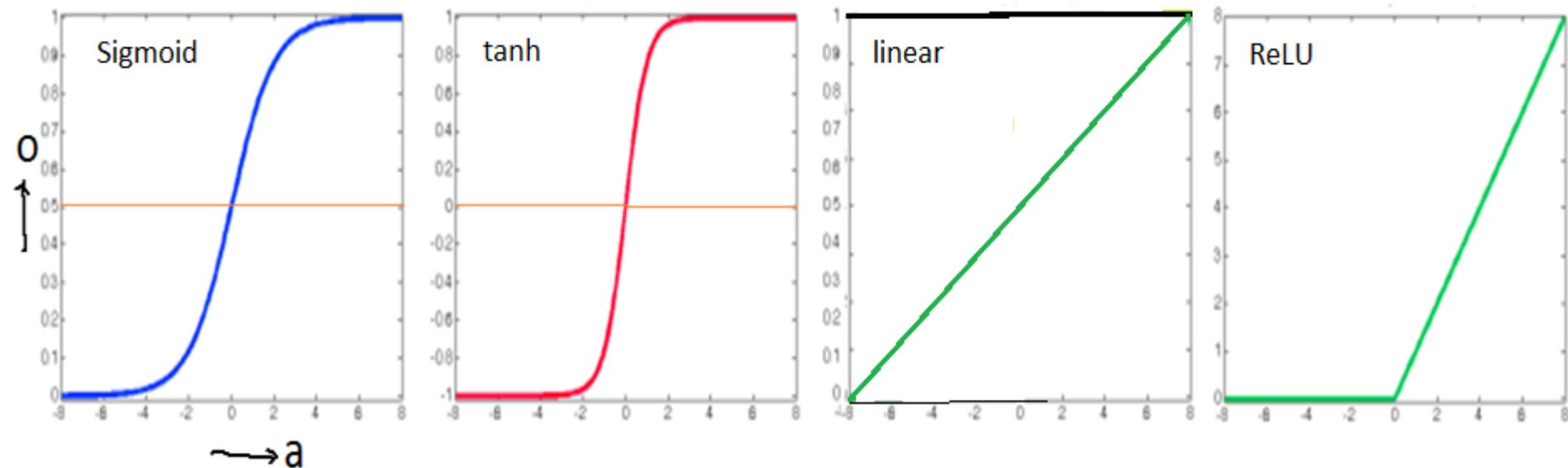
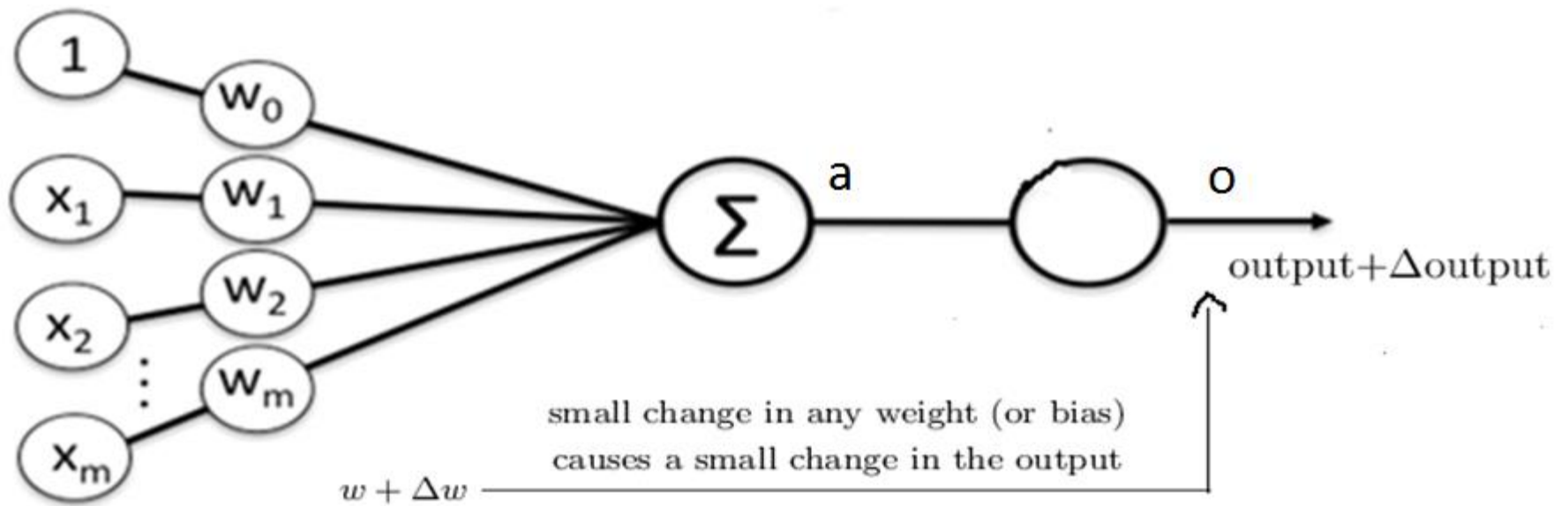
Issues of Perceptron-I

- **Learning is not smooth:** small changes in inputs cause either no change or large change in output and might disturb the patterns already learnt



Solution: Use smoother activation functions instead of step function

Smother activation functions



Smoother activation functions

- Smoother activation functions creates a perceptible change in the output for weight adjustment, which causes minimal disturbance to patterns already learnt
- Some of the smoother activation functions (like sigmoid,tanh,etc.,) also provides better interpretation of output
- The purpose of each activation function will be discussed later in our course

Issues of Perceptron-II

- **Computing Gradient takes longer time:** If the dataset is very large then It can take several hours to compute a single gradient of the error over dataset.

Solution: Use the stochastic approximation of the gradient using a single sample or a group of samples. It has faster convergence rate and having chance of avoiding local minima.

Issues of Perceptron-III

- **Learning is slower:** The perceptron learning is slower due to squared error for sigmoid perceptrons

Solution: The squared error objective may lead to slower learning in case of very bad errors. Use cross-entropy objective function to make faster learning.

Issues with squared loss: slower learning

- What happens if $y = 1$ and $o=0.1$ with initial random weights?
- What happens if $y = 0$ and $o=0.9$ with initial random weights?
- The weight increment will be very small since $o(1-o)$ gives very small multiplier. But in reality, for big errors the improvement is expected to be much higher.

cross-entropy objective function

$$E = -\frac{1}{N} \sum_{n=1}^N [y^{(n)} \log o^{(n)} + (1 - y^{(n)}) \log(1 - o^{(n)})]$$

Intuitive meaning

- Since the output is always between 0 to 1, the above error is always positive
- The cost tends to be close to zero if the output is approaching with actual value

Issues of Perceptron-IV

- **Can only perform binary classification:** The perceptron can be applied to only binary classification task.

Solution: Use single layer of perceptrons for multi-class classification task.

Issues of Perceptron-V

- **Can only detect linear patterns:** The perceptron can't find non-linear boundary for classification task and can't find non-linear function for regression task

Solution: Use multiple cascaded layers of perceptrons to detect non-linear patterns (i.e., to achieve non-linear boundary or function)